

LinSuite Deployment Runbook

Linges Consulting

September 29, 2026

LinSuite Deployment Runbook

This is the one document an operator follows to provision, secure, recover and wind down a LinSuite tenant, and the templates a business needs for its own legal obligations (PIPEDA breach reporting, retention). There is no infrastructure-as-code (`docs/tech-stack.md` §8): every step here is meant to be run by hand, once per tenant, from a clone of this repository.

Conventions used below:

- `<...>` marks a value you supply (a tenant hostname, a bucket name).
- Commands assume you are in the repository root unless a `cd` is shown.
- `.env` is the per-deployment secrets file (`cp .env.example .env`, `chmod 600 .env`) — every variable named below lives there; see `.env.example` for the authoritative list and what each one protects.
- `DATABASE_URL_MIGRATE` is the schema-owner DSN (same one Alembic uses). `DATABASE_URL_PURGE` is the purge role. Neither is the app's own `DATABASE_URL`. Use the right one — the database itself will refuse the wrong one (ADR-0001).

Contents

1. [Provisioning](#)
 2. [First-run setup](#)
 3. [Key escrow](#)
 4. [On-prem TLS](#)
 5. [DMARC for the sending domain](#)
 6. [Offsite backups: Object Lock and the no-delete proof](#)
 7. [Monthly prune, from the operator workstation](#)
 8. [Quarterly restore rehearsal](#)
 9. [Healthcheck / dead-man monitoring](#)
 10. [Dropping an old access-log partition](#)
 11. [Breach register and notification templates](#)
 12. [Per-tenant provisioning checklist](#)
-

1. Provisioning

DIGITALOCEAN

1. Create a Droplet in a Canadian region (Toronto or Montreal, for data residency — `docs/tech-stack.md` §8), Ubuntu LTS, sized for the tenant's expected load. Attach a Volume if you expect to grow disk later; resizing it is a console action, not a rebuild.
2. Install Docker Engine and the Compose plugin on the Droplet.
3. Clone this repository at a known-good tag/commit onto the Droplet (there is no CD pipeline or registry push — `docs/tech-stack.md` §12 — so the Droplet builds the images itself):

```
git clone https://github.com/linges-consulting/LinSuite.git
cd LinSuite
git checkout <release tag or commit>
```

4. `cp .env.example .env && chmod 600 .env`, then fill in every secret (§3 covers escrow) and set `APP_HOST=<tenant-domain>`.
5. DigitalOcean's own Volume encryption (LUKS, AES-256) already covers disk-at-rest (`docs/tech-stack.md` §8 table) — no extra step here.
6. Point the tenant's DNS `A` / `AAAA` record at the Droplet's IP before doing TLS (§4 — DNS-01 needs it resolvable to issue a certificate).
7. `docker compose up -d`.
8. Continue at [§2 First-run setup](#).

ON-PREM

1. Clone the repository onto the customer's own server the same way as above.
2. **Disk encryption is the customer's responsibility here** — there is no automatic equivalent of DigitalOcean's Volume encryption. Enable full-disk encryption (LUKS on Linux, BitLocker on Windows Server if that's the host) *before* installing Docker and bringing up the stack. Without it, the only protection on data at rest is the application-level PDF encryption (`DOCUMENT_MASTER_KEY`) — real, but not a substitute for disk encryption if the drive itself is stolen.
3. `cp .env.example .env && chmod 600 .env`, fill in secrets (§3), set `APP_HOST` to whatever hostname the clinic's LAN resolves to (or leave `localhost` for a single-machine install).
4. Plan disk capacity against a physical drive, not an elastic volume — there is no “resize from the console” here (`docs/tech-stack.md` §8 table).
5. `docker compose up -d`.

6. **Confirm `beat` is actually running:** `docker compose ps` must show the `beat` service `Up`. It is the only thing that creates next year's access-log partition before the app itself restarts (`docs/tech-stack.md` §8 table) — on-prem has no orchestrator to restart it if it crashes, so this is worth checking again a few days after go-live.
7. Continue at [§2 First-run setup](#), then [§4 On-prem TLS](#) before the clinic starts using it over the LAN.

BOTH

```
docker compose ps      # db, redis: healthy. app, worker, beat, frontend, traefik, backup: Up.
```

If `app` keeps restarting, check `docker compose logs app` first — the most common cause is a placeholder secret still in `.env` (`Settings` refuses to boot with one; the error names which variable and the exact `openssl rand -hex 32` command to fix it).

2. First-run setup

1. Retrieve the setup token, either from the boot log:

```
docker compose logs app | grep -A1 "Setup token"
```

or, if that has scrolled past, from the file it's also written to (`SETUP_TOKEN_FILE`, `.env.example`, default `/var/lib/linsuite/setup-token`, inside the `app` container):

```
docker compose exec app cat /var/lib/linsuite/setup-token
```

A restart does not invalidate this token (`auth/setup.py`) — only completing setup does.

2. Open `http://<APP_HOST>/` (or `https://` once §4 is done). The frontend calls `GET /api/setup/status` and shows the wizard while `businesses.setup_completed_at` is null.
3. Fill in: the business name, the business's IANA timezone (`GET /api/setup/timezones` lists the canonical set — get this right the first time; appointment storage semantics depend on it and retrofitting is painful, `docs/tech-stack.md` §17), the first Administrator's email and a password (12-character minimum, breach-screened if `BREACH_CHECK_ENABLED=true`), and the token from step 1.

4. Submit. A wrong or missing token gets `403 INVALID_SETUP_TOKEN`; success stamps `businesses.setup_completed_at` and destroys the token row and file in the same transaction. `POST /api/setup` now answers `404` permanently, across restarts, for every process — there is nothing further to lock down.
5. Log in as the new Administrator. Branding (logo, favicon, colours) is configured from Settings from here; TOTP enrolment is prompted at this account's first login once MFA is set up — neither belongs in the wizard itself.
6. Set the business's retention profile (`regulated_health` or `general_business`, `docs/adr/0001-retention-expiry-and-purge-authority.md` §1) from the admin panel if it was not asked at setup for this release. Get this right before real client data is entered — it governs how long records are kept and cannot be silently changed later without an explicit decision.

3. Key escrow

WHY TWO COPIES, AND WHERE THEY MAY NEVER BE

- **The business holds a sealed, printed copy.** If the vendor disappears or is unreachable, the business can still recover its own data.
- **The vendor holds a second copy, offline, in a per-tenant password-manager vault entry** — not synced to a laptop that also has customer-facing internet access, not in the same password manager entry as unrelated tenants. If the business loses its sheet, the vendor can still help recover.
- **Neither copy lives on the tenant server outside `.env`, and neither lives in the restic repository.** `.env` is mode `0600` and never leaves the server; the restic repository is encrypted with `RESTIC_PASSWORD`, a *different* secret from the ones it protects — anyone holding both the repository and its password would otherwise have plaintext health records the moment they also found the escrow sheet in the same place.

WHAT'S ON THE SHEET

Every secret below lives in `.env` (`.env.example` is the source of truth for exact names).

<code>.env</code> variable	Protects	Rotatable?
<code>DOCUMENT_MASTER_KEY</code>	Every client's document key (<code>customer_document_keys</code>) and the business financial-document key (<code>business_document_keys</code>) — ten years of consents, chart notes, invoices	No — not in v1. Losing or changing it makes every existing document permanently unreadable ciphertext. There is no re-wrap path (ADR-0003 §6; <code>.env.example</code>).
<code>MFA_ENCRYPTION_KEY</code>	Every enrolled TOTP secret	No rotation lever. Losing it means re-setting MFA for every enrolled user.
<code>NOTIFICATION_CREDENTIAL_KEY</code>	The business's stored Resend/SMTP credentials (<code>businesses</code> row)	No rotation lever, same shape as the MFA key.
<code>JWT_SECRET</code>	Session cookie signatures	Yes — the low-stakes one. Rotating it just logs everyone out immediately; it is the documented emergency lever, not a disaster if changed. Escrow it anyway so a lost <code>.env</code> doesn't also mean "everyone re-authenticates with no warning and nobody remembers why."
<code>RESTIC_PASSWORD</code>	The offsite backup repository's client-side encryption	No rotation lever restic makes easy; treat as fixed for the tenant's life.
<code>POSTGRES_PASSWORD</code>	The schema-owner role (<code>linsuite</code>)	Can be changed (<code>ALTER ROLE ... PASSWORD</code>) but touches <code>DATABASE_URL_MIGRATE</code> and the db init script; escrow it rather than plan on rotating casually.
<code>APP_DB_PASSWORD</code>	<code>linsuite_app</code>	Same as above.
<code>PURGE_DB_PASSWORD</code>	<code>linsuite_purge</code>	Same as above.

<code>.env</code> variable	Protects	Rotatable?
<code>BACKUP_DB_PASSWORD</code>	<code>linsuite_backup</code>	Same as above.

FINGERPRINTING WITHOUT EXPOSING THE SECRET

SHA-256 fingerprints let both parties later verify their copy matches what's actually in `.env`, without either copy ever being read aloud, screenshotted or typed into a chat. Run this on the tenant server, once per secret, and write only the fingerprint on the sheet:

```
grep '^DOCUMENT_MASTER_KEY=' .env | cut -d= -f2- | tr -d '\n' | openssl dgst -sha256
```

Substitute the variable name for each row. The pipeline never puts the secret itself on a terminal line by itself — only `openssl dgst` sees the raw bytes, and it prints a digest.

To fingerprint all nine in one pass (still nothing but digests hits the screen):

```
for var in DOCUMENT_MASTER_KEY MFA_ENCRYPTION_KEY NOTIFICATION_CREDENTIAL_KEY
JWT_SECRET \
    RESTIC_PASSWORD POSTGRES_PASSWORD APP_DB_PASSWORD PURGE_DB_PASSWORD
BACKUP_DB_PASSWORD; do
    fp=$(grep "^${var}=" .env | cut -d= -f2- | tr -d '\n' | openssl dgst -sha256 |
awk '{print $2}')
    printf '%-28s %s\n' "$var" "$fp"
done
```

Tenant (business legal name): _____
Deployment host / APP_HOST: _____
Date created: _____
Created by (vendor operator): _____
Witnessed by (business owner/admin): _____

DOCUMENT_MASTER_KEY (v1: cannot be rotated – see note below)

NOTIFICATION_CREDENTIAL_KEY

RESTIC_PASSWORD

APP_DB_PASSWORD (linsuite_app)

BACKUP_DB_PASSWORD (linsuite_backup)

Business copy: sealed in an envelope, signed across the seal by

```

both parties, held by the business off the server
(a safe, or with counsel), never inside the
LinSuite deployment itself.
Vendor copy:  stored offline in the vendor's per-tenant
               password-manager vault entry, not synced to any
               machine with general internet access.
=====

```

4. On-prem TLS

Decide which of these three applies, in this order:

1. **The clinic has a public domain it controls DNS for** → DNS-01 via Traefik's ACME (preferred: fully automatic issuance and renewal, no certificate ever touches disk by hand).
2. **No public domain (a LAN-only install)** → an internal CA, openssl commands below, plus installing the root certificate on each device.
3. **The clinic already runs its own CA** → drop their issued certificate into the same slot internal-CA certificates use; no Traefik reconfiguration needed.

All three ultimately deliver a certificate to the same place; only how it gets there differs.

THE MECHANISM (2) AND (3) BOTH USE

`infra/compose.yaml`'s `traefik` service:

- Listens on `websecure` (`:443` , `TRAEFIK_HTTPS_PORT` in `.env` , default `443`) in addition to `web` (`:80`). TLS termination is configured at the **entrypoint** level (`--entrypoints.websecure.http.tls=true`), not per-router — a per-router `tls` label would make that router refuse the plain `web` entrypoint too, which would break local dev and any on-prem box that hasn't done its TLS step yet. `web` keeps working unchanged either way.
- Reads `infra/traefik/dynamic/tls.yml` (a Traefik file-provider config, bind-mounted read-only) that points at `/certs/server.crt` and `/certs/server.key`.
- `/certs` is `../certs` from the repo root — i.e. `certs/` at the repository root — bind-mounted read-only. It is gitignored: nothing here is ever committed.
- With `certs/` **empty (the default — this is what `docker compose up` gives you with zero TLS configuration)**, Traefik logs one provider error for the missing files and falls back to its own generated self-signed certificate for `websecure`. The stack still starts, `web` still works, and `https://` also answers — just with a certificate nothing trusts yet. This is deliberate: bringing up the stack must never depend on TLS material existing.

- To install a certificate: put `server.crt` (the full chain, if there are intermediates — concatenate leaf then intermediates then root is *not* required, leaf + intermediates is) and `serv-er.key` into `certs/`, then `docker compose restart traefik`. The file provider also hot-reloads on change (`--providers.file.watch=true`), so a restart is a formality, not a requirement.

(1) DNS-01 VIA TRAEFIK ACME

This needs per-tenant DNS provider credentials, so it is not baked into the committed `infra/compose.yaml` — add a small Compose override for this tenant (kept in the tenant's own ops notes/repo, never in this repository) with:

```
services:
  traefik:
    command:
      - --certificatesresolvers.tenant.acme.dnschallenge=true
      - --certificatesresolvers.tenant.acme.dnschallenge.provider=<provider>    #
e.g. cloudflare, route53, digitalocean
      - --certificatesresolvers.tenant.acme.email=ops@<vendor-domain>
      - --certificatesresolvers.tenant.acme.storage=/letsencrypt/acme.json
    volumes:
      - letsencrypt:/letsencrypt
volumes:
  letsencrypt:
```

Apply it alongside the base file: `docker compose -f infra/compose.yaml -f infra/compose.dns01.yaml up -d`.

Then add the DNS provider's own credential environment variables to `.env` (the exact names are the DNS provider's — e.g. Cloudflare wants `CF_DNS_API_TOKEN`; check Traefik's ACME DNS provider reference for the one you're using), and point the existing app/auth/public/frontend routers at this resolver by adding one label per router in the same override:

```
labels:
  traefik.http.routers.app.tls.certresolver: tenant
```

Traefik requests a certificate via a DNS TXT challenge on first boot and renews automatically from then on — nothing in `certs/` is used or needed for this path.

(2) INTERNAL CA — EXACT COMMANDS

Run these from an operator workstation (or the tenant server itself, if you'd rather generate in place — either way, the *root key* (`root.key`) must end up somewhere safe and durable, **not** inside `certs/` on the tenant server, since only the leaf certificate and its key belong there).

```
# One-time per tenant (or per vendor, if you'd rather run one CA across several
LAN-only
# tenants — a 10-year root is long-lived on purpose, so this step happens
rarely):
openssl genrsa -out root.key 4096
openssl req -x509 -new -nodes -key root.key -sha256 -days 3650 \
  -subj "/C=CA/ST=<Province>/O=<Clinic Name> Internal CA/CN=<Clinic Name>
Internal CA" \
  -out root.crt

# Per-server leaf certificate. 825 days (~27 months) is the ceiling current
browsers and
# OSes accept without complaint — set a calendar reminder at day 795 to reissue.
openssl genrsa -out server.key 2048
cat > server.ext <<EOF
subjectAltName = DNS:<tenant-hostname>, DNS:localhost, IP:<server-LAN-IP>
EOF
openssl req -new -key server.key \
  -subj "/C=CA/ST=<Province>/O=<Clinic Name>/CN=<tenant-hostname>" \
  -out server.csr
openssl x509 -req -in server.csr -CA root.crt -CAkey root.key -CAcreateserial \
  -out server.crt -days 825 -sha256 -extfile server.ext
```

Install it into the stack:

```
mkdir -p certs
cp server.crt server.key certs/
docker compose restart traefik
```

Verify (from any machine that can reach the server, using the root certificate you just made — never the private key):

```
curl --cacert root.crt https://<tenant-hostname>/api/health
```

or, to inspect the served certificate directly:

```
openssl s_client -connect <tenant-hostname>:443 -servername <tenant-hostname>
</dev/null 2>&1 \
| openssl x509 -noout -subject -issuer -dates
```

Verified locally: this exact sequence (10-year root, 825-day leaf with the SAN extension above, `certs/` + `docker compose restart traefik`) was run against this repository's compose stack; `curl --cacert root.crt https://localhost/api/health` returned `200` with the generated certificate's subject and issuer matching, and `http://localhost/api/health` kept answering `200` throughout — the `web` endpoint is unaffected by installing a `websecure` certificate.

Device install steps

Distribute `root.crt` **only** — never `root.key` (that stays with whoever holds the CA; a leaked root key means every certificate it has ever issued, or ever will, is worthless the moment someone notices).

Windows (per machine, needs an elevated prompt):

```
certutil -addstore -f "ROOT" root.crt
```

For more than a handful of machines, distribute via Group Policy instead: *Computer Configuration* → *Windows Settings* → *Security Settings* → *Public Key Policies* → *Trusted Root Certification Authorities* → Import `root.crt`.

macOS:

```
sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.key-chain root.crt
```

Or via the GUI: double-click `root.crt` to open it in Keychain Access, add it to the *System* keychain, then double-click the entry, expand *Trust*, and set “When using this certificate” to *Always Trust*.

iPad / iOS:

1. Get `root.crt` onto the device (AirDrop, or email it to an account only clinic staff use).
2. Open it, then *Settings* → *General* → *VPN & Device Management* and install the profile it offers.

3. This step is easy to miss and the certificate silently won't be trusted without it:

Settings → *General* → *About* → *Certificate Trust Settings*, then enable full trust for the new root certificate. iOS installs a certificate as merely *known* until this second step explicitly trusts it for TLS.

(3) THE CLINIC'S OWN CA

The clinic's IT department issues a certificate and key for the tenant's hostname from their existing PKI. Ask for the leaf certificate (plus any intermediate certificates their CA uses — concatenate leaf then intermediates into one `server.crt`, in that order) and the private key. Install exactly as in (2)'s "Install it into the stack" step:

```
cp server.crt server.key certs/
docker compose restart traefik
```

No device install step is needed here — the point of this path is that clinic devices already trust their own CA.

5. DMARC for the sending domain

Do this only after the business's outgoing email is actually configured and verified — DMARC has nothing meaningful to enforce until SPF/DKIM alignment exists to check.

1. In LinSuite, *Settings* → *Notifications*: set the sender to Resend, set `resend_from_address` to an address at the domain you're about to configure (e.g. `notifications@<clinic-domain>`), and use the in-app "send test email" action — this is what sets `businesses.resend_domain_verified_at`.
2. In Resend's own dashboard, add and verify that sending domain. Resend gives you the SPF and DKIM DNS records to add at your DNS host; add exactly what it shows and wait for it to show verified there too — this step is what actually makes SPF/DKIM pass, not the in-app test send.
3. Add a DMARC record, starting at `p=none` (monitor only — nothing is rejected or quarantined, but you start collecting aggregate reports of who is and isn't passing alignment):

```
_dmarc.<clinic-domain>. IN TXT "v=DMARC1; p=none; rua=mailto:dmarc-reports@<vendor-or-clinic-domain>; fo=1"
```

4. Watch the aggregate reports (`rua`) for **two consecutive clean weeks** — no legitimate mail failing DKIM or SPF alignment. Then tighten:

```
_dmarc.<clinic-domain>. IN TXT "v=DMARC1; p=quarantine; rua=mailto:dmarc-reports@<vendor-or-clinic-domain>; fo=1"
```

5. Moving to `p=reject` after that is a reasonable later step but isn't required by this runbook — treat it as an operator judgement call once `quarantine` has run clean for a while too.

6. Offsite backups: Object Lock and the no-delete proof

BUCKET SETUP (BACKBLAZE B2 — THE DOCUMENTED OFFSITE TARGET, `DOCS/TECH-STACK.MD §10`)

1. Create a private bucket with File Lock **enabled at creation** (Backblaze's console: Buckets → Create a Bucket → toggle *File Lock*; check whether your `b2` CLI version can enable it on an existing bucket instead — `b2 bucket update --help` — but enabling it at creation is the option that always works).
2. Set the bucket's default file retention to **Governance mode**, 30–90 days (Governance, not Compliance: it still blocks the no-delete application key below exactly the same way, but leaves the account owner a documented emergency-bypass path if one is ever genuinely needed, which a self-inflicted Compliance-mode lockout does not):

```
b2 bucket update <bucket-name> --default-retention-mode governance --default-retention-period "60 days"
```

(exact flag names vary by installed `b2` CLI version — confirm with `b2 bucket update --help`; 60 days is a reasonable default inside the ticket's 30–90 day window.)

3. Create the **write-and-read, no-delete** application key the tenant server will actually use, scoped to this one bucket:

```
b2 key create --bucket <bucket-name> linsuite-<tenant>-backup
readFiles,writeFiles,listFiles,listBuckets
```

Note what's *not* in that capability list: `deleteFiles`. This key cannot issue a delete or hide call the server will honour, full stop — this is the primary protection (`docs/tech-stack.md §10`), and it holds even before Object Lock is considered.

4. Put the key into `.env`:

```
BACKUP_TARGET=b2
RESTIC_REPOSITORY=b2:<bucket-name>:<tenant-path>
B2_ACCOUNT_ID=<keyID>
B2_ACCOUNT_KEY=<applicationKey>
RESTIC_PASSWORD=<already escrowed per §3>
```

5. First deploy only, initialise the (empty) repository:

```
docker compose run --rm -e RESTIC_INIT=1 backup backup.sh
```

then set `RESTIC_INIT=0` back in `.env` — leaving it at `1` would silently start a fresh, unrelated repository history if the real one ever became unreachable, instead of failing loudly (`infra/backup/backup.sh`’s own comment on this).

THE NO-DELETE PROOF (DO THIS ONCE PER TENANT; RECORD THE OUTPUT ON THE CHECKLIST, §12)

Proof 1 — the tenant server’s own key cannot prune. From the tenant server, using the restricted key from step 3 above:

```
docker compose exec backup restic -r "$RESTIC_REPOSITORY" forget --prune --keep-last 1
```

This **must fail** with a permission-denied error from B2 — the key holds no `deleteFiles` capability. If this succeeds, something is misconfigured (the key was created with the wrong capability list, or `.env` has a different, more privileged key in it) — fix it before trusting this backup.

Proof 2 — Object Lock refuses deletion even with a delete-capable key, inside the window. From an operator workstation, using a *separate* administrative B2 key that *does* have `deleteFiles` (never install this key on the tenant server — §7 covers exactly this key), pick one recent object and try to delete it before its retention window has passed:

```
b2 file delete <fileName> <fileId>
```

This **must also fail**, with an Object Lock / “not allowed to delete or hide files” error, until the retention period on that specific object passes — this is what actually protects the backups even against a fully compromised, delete-capable credential, which the no-delete key alone does not

(a stolen *admin* key would otherwise still be able to prune everything).

Record both commands' output and today's date on the provisioning checklist (§12) — this is manual, recorded evidence, not something CI re-proves on every PR.

7. Monthly prune, from the operator workstation

Pruning removes old snapshots per a retention policy — something the tenant server's own key cannot do (§6), by design. Run this from an operator's own machine, once a month, using a *separate*, delete-capable B2 application key that **never** exists in the tenant's `.env` or anywhere on the tenant server:

```
export RESTIC_REPOSITORY=b2:<bucket-name>:<tenant-path>
export RESTIC_PASSWORD=<escrowed restic password>
export B2_ACCOUNT_ID=<operator's own delete-capable key id>
export B2_ACCOUNT_KEY=<operator's own delete-capable key>

restic snapshots                                # sanity
check first
restic forget --keep-daily 30 --keep-weekly 8 --keep-monthly 12 --prune
```

Record the date on the provisioning checklist (§12). If `restic forget --prune` ever succeeds using the credentials that live in the tenant's own `.env`, stop and treat it as an incident — see Proof 1 in §6.

8. Quarterly restore rehearsal

On every PR, CI already proves the mechanism works end to end against a throwaway local repository (`.github/workflows/ci.yml`'s `backup-rehearsal` job, running `infra/backup/rehearse.sh`): a fresh Postgres, migrated, seeded with a customer and a sealed document, backed up with the real `backup.sh` inside the real backup image, restored into a second database, row counts and the document's ciphertext + digest compared (`infra/backup/compare.py`), then `customers.tasks.purge_expired` run against the restored copy (ADR-0001 rule 9).

Quarterly, and once at initial provisioning, run the same rehearsal against the tenant's *real* offsite repository, from an operator workstation — not CI's throwaway local one. The simplest way is to point `infra/backup/rehearse.sh`'s environment at the real target instead of `local`

(read the script — it's meant to be adapted this way; the parts worth keeping unchanged are the seed/compare/purge steps, and the part to repoint is the `BACKUP_TARGET` / `RESTIC_REPOSITORY` / credentials the backup container uses). At minimum, by hand:

1. Restore the latest snapshot from the real repository into a scratch database (same `pg_restore` step `rehearse.sh` runs).
2. Compare row counts and one document's digest against a known-good original (`infra/backup/compare.py <original-dsn> <restored-dsn> <document-id>`).
3. Run the post-restore purge against the restored copy — the documented recovery step is the one that must actually be tested:

```
docker compose exec worker celery -A core.celery_app call
customers.tasks.purge_expired
```

or, run inline the way `rehearse.sh` does against a scratch database with its own env:

```
uv run python -c "from customers.tasks import purge_expired;
print(purge_expired.run())"
```

4. Tear down the scratch database.

Record the date, pass/fail, and the counts from step 2 on the provisioning checklist (§12).

9. Healthcheck / dead-man monitoring

1. Create two checks — one for maintenance, one for backup — at healthchecks.io (or a self-hosted instance: https://github.com/healthchecks/healthchecksSMARTPANTS_PRE-SERVED_736). Set each check's expected period/grace so you're alerted **after 36 hours without a success ping** (maintenance runs nightly at 03:15/03:30 UTC; backup runs at `BACKUP_SCHEDULE`, default `0 2 * * *` — 02:00 UTC).
2. Set in `.env`:

```
HEALTHCHECK_URL_MAINTENANCE=https://hc-ping.com/<maintenance-check-id>
HEALTHCHECK_URL_BACKUP=https://hc-ping.com/<backup-check-id>
```

Both are optional (`core/config.py`, `infra/backup/backup.sh`) — leaving either unset just means that job never makes the request; a deployment with no monitoring configured still runs normally. Set them anyway; this is what actually catches a scheduler or worker that quietly stopped.

3. [docker compose up -d beat worker backup](#) to pick up the new env.
4. [Confirm each ping actually fires:](#)

```
docker compose exec worker celery -A core.celery\_app call core.tasks.maintain\_partitions
docker compose exec backup /usr/local/bin/backup.sh
```

then check the [healthchecks.io](#) dashboard shows a green “success” for both within a minute or two. [core.tasks.maintain_partitions](#) and [customers.tasks.purge_expired](#) both ping [HEALTHCHECK_URL_MAINTENANCE](#) on success ([core/healthcheck.py::ping_maintenance](#)); [backup.sh](#) pings [HEALTHCHECK_URL_BACKUP](#) on success and [\\${HEALTHCHECK_URL_BACKUP}/fail](#) on any error.

[/api/health](#)’s [partitions](#) field ([ok](#) / [next_year_missing](#)) is a second, cheaper signal for the same failure mode — worth pointing an uptime monitor at too, though it never fails the health check’s status code on its own (a missing future partition is not “the app is down”).

10. Dropping an old access-log partition

This is deliberately **never automated**. No runtime role can remove access-log history: the app role only inserts, and since migration 0067 the purge role holds [SELECT](#) alone on [audit_access_log](#) and every partition [public.ensure_access_log_partitions\(\)](#) creates (ADR-0001 rule 15). Dropping a whole partition is DDL, and DDL on this table is schema-owner-only — so trimming the audit trail is a deliberate, documented act by the operator, not something a nightly job decides on its own.

1. [Confirm nothing in the partition is still inside anyone’s retention window](#) before touching it. The access log inherits the business’s longest applicable retention horizon ([docs/tech-stack.md](#) §16 — ten years from the last entry for [regulated health businesses](#); [general_business](#) tenants have no such statutory floor but honour deletion requests promptly rather than on a fixed schedule). Check the partition’s actual date range first, using the schema-owner DSN:

```
psql "\$DATABASE\_URL\_MIGRATE" -c \
"SELECT min\(occurred\_at\), max\(occurred\_at\), count\(\*\) FROM
audit\_access\_log <year>;"
```

2. **Detach, then drop — two separate statements**, not `DROP TABLE ... CASCADE` on the parent, so there is a point between them where the detached table could still be archived or exported if you change your mind:

```
psql "$DATABASE_URL_MIGRATE" -c \
    "ALTER TABLE audit_access_log DETACH PARTITION audit_access_log_<year>;"
psql "$DATABASE_URL_MIGRATE" -c \
    "DROP TABLE audit_access_log_<year>;"
```

3. Use `DATABASE_URL_MIGRATE` — the schema-owner DSN, the same one Alembic uses — never the app or purge DSN. Neither of the runtime roles may run this DDL, by design.

11. Breach register and notification templates

PIPEDA requires reporting a breach with a “real risk of significant harm” to the Privacy Commissioner and to affected individuals, and requires a record of **every** breach — not only reportable ones — retained for 24 months ([docs/tech-stack.md](#) §16). These are obligations on the *business*, not something the software automates; LinSuite’s own audit log (`audit_events`, and `audit_access_log` for reads — see the access-log report at `GET /api/admin/customers/{id}/access-log`, behind `audit.view`, Admin Mode) is what makes scoping a breach (“which records were actually exposed”) tractable in the first place.

BREACH REGISTER

A starting CSV template lives at [docs/runbook/breach-register-template.csv](#) — copy it per tenant and keep it retained for 24 months from each row’s `date_discovered`. Columns:

```
date_discovered, date_occurred, description, records_affected_estimate, data_elements_involved, cause, containment_actions, risk_assessment, commissioner_notified_date, individuals_notified_date, notification_method, retain_until, notes
```

NOTIFICATION TO THE PRIVACY COMMISSIONER OF CANADA

To: OPC Breach Reporting <PIPEDA.Compliance-Conformite.LPRPDE@priv.gc.ca>
 Subject: Breach of Security Safeguards Report – <Business Legal Name>

1. Organization
 Name: <Business Legal Name>
 Contact: <name, title, phone, email>
 2. Description of the breach
 Date/time period the breach occurred: <...>
 Date/time discovered: <...>
 Description of what happened: <...>
 Cause (if known): <human error / system failure / malicious action / other>
 3. Personal information involved
 Number of individuals affected (or best estimate): <...>
 Types of personal information involved: <e.g. name, contact details, health record details, financial/billing information>
 Was the information encrypted or otherwise protected: <...>
 4. Assessment of risk of significant harm
 <Explain the factors considered: sensitivity of the information, probability it has been/will be misused, and any other relevant circumstances – per PIPEDA's "real risk of significant harm" standard.>
 5. Steps taken to reduce the risk of harm / prevent recurrence
 <...>
 6. Steps taken, or planned, to notify affected individuals
 Method: <...>
 Timing: <...>
 Content: <attach or summarize the individual notification below>
 7. Whether any other organizations were notified (e.g. law enforcement, other regulators)
 <...>
- Submitted by: <name, title>
 Date: <...>

NOTIFICATION TO AN AFFECTED INDIVIDUAL

Send this **per individual**, through LinSuite's own notification engine (Settings → Notifications' configured sender) — never as a bulk message that exposes one recipient's details to another.

Subject: Important notice about your personal information at <Business Name>

Dear <Client First Name>,

We are writing to let you know about a privacy incident that may have affected your personal information held by <Business Name>.

What happened:

<Plain-language description – when, what happened, how it was discovered.>

What information was involved:

<Specifically what of theirs – e.g. "your name, contact information, and appointment history" – never more detail than is true, never vaguer than is honest.>

What we are doing about it:

<Containment and remediation steps already taken, and what's still in progress.>

What you can do:

<Concrete, individual steps – e.g. monitoring for unusual activity, contacting a credit bureau if financial information was involved, changing a password if credentials were involved.>

If you have questions, please contact:

<Name, role, phone, email – a real person who can actually answer.>

You also have the right to contact the Office of the Privacy Commissioner of Canada if you

have concerns about how this was handled: <https://www.priv.gc.ca>

Sincerely,

<Name, title, Business Name>

12. Per-tenant provisioning checklist

Copy this per tenant and keep it filed with that tenant's escrow sheet.

LINSUITE PROVISIONING CHECKLIST – <Tenant>

```

=====
[ ] Stack provisioned (§1) – DigitalOcean / on-prem: _____
[ ] First-run setup completed (§2) – date: _____, admin email: _____
[ ] Key escrow sheet completed, both copies placed (§3) – date: _____
[ ] On-prem TLS configured (§4), if applicable
    Path used: [ ] DNS-01 [ ] Internal CA [ ] Clinic's own CA [ ] N/A (DO +
ACME)
    Certificate expiry date: _____
    Next renewal/reissue reminder set for: _____
[ ] DMARC configured (§5)
    p=none set on: _____ p=quarantine set on: _____ (after 2
clean weeks)
[ ] Object Lock bucket + no-delete key set up (§6) – date: _____
    Proof 1 (server key cannot prune) – ran _____, result: _____
    Proof 2 (Object Lock blocks delete inside window) – ran _____, result:
_____
[ ] Monthly prune schedule established (§7) – reminder set: [ ] yes
    Prune log:
        _____ (date) result: _____
        _____ (date) result: _____
[ ] Healthcheck monitoring configured (§9) – date: _____
    Maintenance check id: _____ Backup check id: _____
    Test ping confirmed for both: [ ] yes
[ ] Quarterly restore rehearsal log (§8):
    _____ (date) result: _____ row counts match: [ ] yes purge
ran clean: [ ] yes
    _____ (date) result: _____ row counts match: [ ] yes purge
ran clean: [ ] yes
    _____ (date) result: _____ row counts match: [ ] yes purge
ran clean: [ ] yes
    _____ (date) result: _____ row counts match: [ ] yes purge
ran clean: [ ] yes
[ ] Retention profile confirmed with business (regulated_health / general_busi-
ness) – date: _____
[ ] Breach register file created for this tenant (§11), 24-month retention noted
=====

```